

THE COMPLETE RULEBOOK

Every rule to follow · Every habit to build · Every mistake to avoid

CAT 01 PROBLEM SOLVING RULES

rulebook.md

● The 30-minute rule — always

01

Attempt every problem solo for a minimum of 30 minutes before looking at any hint or editorial. Set a physical timer. The struggle IS the training. Skipping it means you only learn to recognise solutions — not find them.

■ After 30 min still stuck: read *ONLY* the first hint, not the full solution. Try 15 more minutes. Then full editorial.

● Never copy-paste any solution — ever

02

Even after reading the full editorial and understanding it completely — close it, open a blank editor, and write the solution from scratch from memory. Copy-paste creates the false feeling of having solved a problem.

■ If you cannot write it from memory after reading the solution, you do not know it yet.

● Write time and space complexity for every problem

03

No exceptions. After every accepted solution write $O(?)$ time and $O(?)$ space in your Problem Log immediately. Interviewers ask this in the first 60 seconds of the follow-up discussion. Make it completely automatic.

● Pseudo-code on paper before coding

04

For every medium and hard problem: write numbered pseudo-code steps on paper before opening your editor. This reduces bugs by at least 50% and mirrors exactly what interviewers want to see — your thinking process, not your typing speed.

● Tag every problem with its core pattern

05

After solving: identify the exact pattern — BFS, DP-Knapsack, Monotonic Stack, Greedy-Interval, Binary Search on Answer, Sliding Window, Two Pointer. Log it in your tracker. After 200 tagged problems you will automatically see patterns in new problems before you even start coding.

● Dry run your code on a hand-made example

06

Trace through your code line by line with your own example before every submission. This single habit will save you from wrong answers on OAs and interview rounds more than anything else you can do.

● Identify the pattern before writing any code

07

Sorted array and search = Binary Search. Connected cells = BFS or DFS. Optimal substructure = DP. Repeating subarray range = Sliding Window. Name the pattern on paper before touching the keyboard. This separates instinctive solvers from beginners.

- **Work through 2 examples by hand first**

08

Take the given example AND create your own example. Trace the expected answer manually on paper. This reveals the correct approach 80% of the time before you even think of any algorithm.

CAT 02

LEARNING RULES

rulebook.md

- **One topic at a time — full mastery before moving forward**

09

Do not jump from Arrays to Trees before hitting your target problem count for Arrays. Topic-hopping feels like progress but is the single biggest reason people have 500 problems solved and still blank out on mediums in actual interviews.

- **Watch → Close → Write from memory → Verify**

10

For every concept video: watch it fully once. Close it completely. Write what you remember in Notebook 1 from memory. Then re-open to fill the gaps you missed. Never write notes WHILE watching — that creates passive learning, not active recall.

- **Solve problems immediately after watching**

11

Watch a topic video then solve 3 problems on that exact topic the same day, in the same session. The connection between concept and application must happen while the concept is still fresh in your working memory.

- **30 minutes maximum per video session**

12

If a video is longer than 30 minutes, split it into two sessions with a problem-solving session in between. Watching 4 hours of tutorials in one sitting retains almost nothing. Time spent actively coding always beats time spent passively watching.

- **Write CS subject answers in your own words only**

13

If you copy GFG text directly into your notebook, you will not recall it under interview pressure. The act of translating concepts into your own language is exactly what creates durable, retrievable memory. If you cannot write it fresh, go back and re-learn.

- **If you cannot explain it in 60 seconds, you do not know it**

14

Test for every CS concept: can you explain it in plain language in under 60 seconds with one concrete example? That is the real bar — not whether you can quote five GFG paragraphs. Interviewers reward clarity and simplicity over textbook completeness.

CONSISTENCY · DAILY · CONTEST

The habits that separate placed candidates from everyone else

CAT 03

CONSISTENCY RULES

rulebook.md

- **Never take 2 consecutive days off**

15

One rest day per week is healthy and necessary. But two consecutive days off breaks the habit loop and the streak psychology. On your hardest days — do just one LeetCode easy problem, 20 minutes. Keep the chain alive. Momentum across a 12-month preparation is everything.

- **Be brutally honest in your tracker**

16

If you saw the full solution — mark it Saw Solution. Do not mark it Needed Hint just to feel better about yourself. A tracker full of honest entries is your real preparation mirror. False entries create dangerous false confidence before interviews.

- **LeetCode Daily — every single day from Phase 3**

17

One LeetCode Daily problem every single day without fail. It rotates through all topics automatically — this is your built-in spaced repetition system. Even on your worst days: just the daily problem. An Amazon and Flipkart hire called this their single most important preparation habit.

- **Weekly review every Sunday — 10 minutes**

18

Fill your Weekly Review every Sunday without fail: problems solved this week, solo percentage, weakest topic, what went well, what to fix next week. Without this weekly review you will spend Month 6 making the exact same mistakes you should have fixed in Month 2.

- **Volume in Phase 1 and 2, quality in Phase 3 and 4**

19

In Phase 1 and 2: solve MORE problems, not perfectly. Speed of exposure to different problem types builds pattern intuition faster than perfecting one solution. In Phase 3 and beyond: depth matters fully — understand every approach, every edge case, and every complexity trade-off.

- **Speak your approach out loud from Phase 3**

20

Practice explaining your thinking as you code from Phase 3 onwards. Solve at least two problems per week out loud, exactly as if you are in a real interview. Interviewers at Google, Amazon, and Flipkart evaluate your thinking process — not just the final code you submit.

CAT 04

DAILY HABITS TO BUILD

rulebook.md

- **Phone face-down during every Pomodoro**

21

Not on silent — face-down on the table. Every single notification during a 40-minute Pomodoro costs you 15 minutes of cognitive re-focus time. Three phone checks in one session means you effectively worked for only five minutes. DSA requires completely uninterrupted deep focus blocks.

● **First problem of every morning goes on paper**

22

Write the problem number, your approach, and your pseudo-code on paper for the very first problem every single morning. This warms up your analytical brain exactly the same way a physical warm-up prevents injury before sport.

● **Log every problem before closing the laptop**

23

Do not batch-log multiple days of problems on Sunday. Log each problem the same day you solved it, while the approach is still fresh in your mind. A problem logged three days later has no useful notes — you will not remember what specifically tripped you up.

● **Notebook entry before sleep — 10 minutes every night**

24

Every single night before sleeping write: (1) the topic you learned today in 2 lines, (2) three key patterns you noticed across today's problems, (3) one specific mistake you made today. Sleep consolidates memory — what you write just before sleep is retained far better than anything else.

● **Sunday test — blank paper, all topics from this week**

25

Every Sunday without exception: for every topic you studied this week, write the full algorithm and the C++ template code on a completely blank page from memory. What you can write from memory is what you truly know. What you cannot write is what still needs more work.

● **Review all today's complexities before closing**

26

At the end of every single session: scroll through today's problems in your tracker and confirm you have written time complexity and space complexity for every single problem you solved. Do not close your laptop until this is done for every problem.

CAT 05

CONTEST RULES

rulebook.md

● **Show up to every contest — no excuses**

27

Missing a contest is exactly the same as skipping a full training session. Even if you are tired, even if you will only solve problem A — you must show up. If you miss a live contest, do it as a virtual contest within 24 hours. The habit of showing up under pressure IS the skill you are building.

● **Read ALL problems in the first 5 minutes**

28

Before solving anything — read every single problem statement quickly first. Sometimes problem C is genuinely easier for you than problem B. Knowing all options lets you prioritize correctly. Most beginners miss easy points by tunnel-visioning on B when C was completely solvable for them.

● **If stuck for 20 minutes — move to the next problem**

29

Do not spend 90 minutes on one problem while leaving two easier problems completely untouched. In contests, solving more problems partially beats solving one deeply. Move on, return if time allows. This is the habit that most separates experienced competitive programmers from beginners.

● **Upsolve every unsolved problem within 24 hours**

30

This is completely non-negotiable. The problems you could not solve during the contest are exactly the problems you need to learn from most. Read the editorial, understand it fully, then code it completely from scratch without looking at any reference.

■ *Mark Upsolved YES in your tracker only after coding it from scratch — not just after reading the editorial.*

● **Write the key lesson from every contest — 1 line**

31

After every single contest: write exactly what pattern you missed or what mistake cost you points. Example: Forgot to mark node as visited before pushing to queue, causing infinite loop. These specific one-line lessons are worth more than 10 hours of tutorial watching.

● **Never care about rating dropping — only what you learned**

32

Your rating will fluctuate throughout your entire preparation — that is completely normal. The only two metrics that actually matter: did you upsolve every problem you could not solve? Did you add the key lesson to your journal? A -50 rating drop that taught you a new pattern is worth more than a +50 gain on familiar problems.

WHAT YOU MUST NEVER DO

Every item on this page is a trap. Recognise it. Avoid it. Every single one.

CAT 06 ABSOLUTE DON'TS — NEVER DO THESE

rulebook.md

● Do not switch programming languages

X

You chose C++. Stay with C++ for the entire 12 months. Switching to Java or Python mid-preparation because it has cleaner syntax wastes weeks of hard-built STL muscle memory. Every single hour spent re-learning syntax is an hour not spent on DSA.

■ *Language consistency = faster typing = more time thinking about algorithms.*

● Do not watch tutorials without solving problems after

X

Watching three hours of DP videos and feeling like you know DP is the single most dangerous trap in DSA preparation. You know absolutely nothing until you have coded it from scratch under pressure. The rule: one hour of watching must be followed by two hours of solving.

■ *Tutorial hell is real. It feels productive every single day. It is not.*

● Do not look at solutions too quickly

X

Looking at a solution at minute 5 because you are not getting it trains your brain to become helpless when faced with difficulty. The discomfort you feel when stuck IS the training. Every single minute of genuine struggle builds real problem-solving muscle that no tutorial can replace.

■ *30-minute minimum before any hint. No exceptions, no negotiation.*

● Do not solve problems randomly without topic focus

X

Picking random LeetCode problems because variety is good builds zero pattern recognition. In Phase 1 through Phase 3 you must solve strictly by topic in the sequence given. Random problem grinding only becomes useful in Phase 4 after solid topic mastery has already been built.

■ *Random grinding produces random unpredictable results. Structured practice produces interview readiness.*

● Do not skip your weekly contest — ever

X

Missing a contest is not a rest day — it is a missed pressure simulation. Online Assessments and technical interviews ARE timed contests performed under real stress. The only way to become comfortable with that exact kind of pressure is to practice it every single week without skipping.

● Do not grind 10 problems one day then rest for 3 days

X

Daily consistency beats weekend marathons every single time without exception. Three problems solved daily for 30 days gives you 90 problems with proper spaced repetition and retention. Ten problems in one day then three days of rest gives you maybe 40 problems actually retained in memory.

■ *Spaced consistent daily practice always beats crammed irregular practice.*

● Do not use social media during study sessions

×

Every single context switch from your problem to Instagram and back costs you 15 to 20 minutes of cognitive re-focus time. Three phone checks in a single 40-minute Pomodoro means you effectively worked for only about five real minutes. Phone goes face-down on the table — not on silent, face-down.

■ *Deep focused work requires zero interruptions. Protect your focus blocks aggressively.*

● Do not mark problems dishonestly in your tracker

×

Marking a problem as Solved Solo when you actually copied the full solution is lying directly to yourself. Your tracker is your preparation mirror. False entries create dangerously false confidence before interviews where accurate self-assessment is completely critical.

■ *Honest tracker entries = accurate self-assessment = correct preparation adjustments.*

● Do not jump to advanced topics before mastering basics

×

Attempting Segment Trees before you have properly mastered Arrays, or diving into Dynamic Programming before you have truly nailed Recursion, means you will understand nothing and retain even less. The roadmap sequence is not arbitrary — it is the only ordering that actually works.

■ *Build the foundation completely before adding the floors above it.*

● Do not memorise solutions — understand the patterns

×

Memorising the specific solution to LeetCode 200 helps you pass only LeetCode 200. Deeply understanding WHY Breadth First Search is the right algorithm for connected component problems helps you correctly solve 50 completely different unseen problems. Companies rotate their questions. Core patterns never change.

■ *Patterns and principles will always beat memorised solutions in real interviews.*

● Do not build 5 half-finished projects

×

Five incomplete repositories on your GitHub profile signals scattered unfocused thinking to every interviewer who looks at your profile. One project built completely and properly — with a clean README, meaningful commits, fully working code, and a demo — is worth more than ten half-done clones of popular apps.

■ *Interviewers DO click your GitHub repos. Depth and completeness always beats quantity.*

● Do not ignore CS subjects until Phase 5

×

Operating Systems, DBMS, and Computer Networks questions are asked IN THE EXACT SAME INTERVIEW ROUND as DSA at Microsoft, Adobe, and D.E. Shaw. Starting CS subjects in Phase 3 gives you six full months of gradual comfortable learning. Starting them in Month 11 gives you only panic.

■ *Allocate one Pomodoro per day to CS subjects starting from Phase 3. No exceptions.*

CAT 07

THINGS TO ALWAYS TAKE CARE OF

rulebook.md

Your 4 Notebooks

Keep all four physical notebooks active and in use at all times: DSA Concepts, Problem Log, CS Subjects Q and A, and Mistakes Journal. Write in them every single day. Read your entire Mistakes Journal cover-to-cover exactly one week before every interview round.

Your GitHub Commit Graph

Push code to GitHub every single day — even if it is just today's three LeetCode solutions organised neatly by topic. The green commit graph on your profile is a silent but powerful signal of discipline and consistency that recruiters and technical interviewers notice and respect.

Your LinkedIn — Post Weekly

Post something every single week without exception: your contest result, a topic you just completed, a milestone you reached, a problem you found particularly interesting. Flipkart and Adobe HRs actively search for and DM students who post consistently. One post per week equals 52 recruiter touchpoints per year.

CF Rating — Track Every Single Contest

Log every single Codeforces contest result in your tracker immediately after it ends. Your target milestones are 1200 rating (Pupil) by Month 6 and 1400 rating (Specialist) by Month 9. Once you consistently reach 1400 plus on Codeforces, LeetCode medium problems start feeling genuinely manageable.

Your Resume — Keep It Updated

Update your resume every single time you hit a meaningful milestone: 100 problems solved, 200 problems, a new completed project, or a new Codeforces rating. Use Jake's Resume template on Overleaf. It is the most widely recommended ATS-friendly one-page LaTeX template. Always include your LeetCode and Codeforces profile links.

Your Project on GitHub

Your one project must have a clean and detailed README, fully working code, clear setup instructions, and ideally a live demo link. You must be able to explain every single line of code you wrote. Practice your two-minute project pitch out loud — you will be asked about your project in every single first interview round without exception.

Weekly Review — Every Sunday, 10 minutes

Every Sunday fill in your Weekly Review without skipping: total problems this week, your solo percentage, your weakest topic, what went well, and what you will fix next week. This ten-minute habit is your course-correction system. Without it you will quietly drift off-track for months before you even notice.

Spaced Repetition — Never Ever Skip It

Revisit every topic you learn at these intervals: one day after learning it, then three days after, then one week after, then two weeks after. Do two cold problems per topic at the end of every month as a test. LeetCode Daily is your automatic built-in daily revision. Without spaced repetition, 70% of what you learn is forgotten within one week.

DSA

= OA filter

CS subjects

= interview survival

Projects

= shortlisting

LinkedIn

= the call

Consistency

= the offer